

Material de Estudo: Estruturas de Dados e C

Edição Atualizada: Conceitos Intuitivos + Fundamentos Técnicos Profundos

Este documento foi reescrito para focar primeiro na **intuição**. Antes de entendermos o código, precisamos entender como essas estruturas existem no mundo real e por que foram inventadas. Só depois traduziremos essa lógica para a Linguagem C.

Módulo 1: Pilhas (Stacks)

A Intuição LIFO (Last-In, First-Out)

A forma mais fácil de compreender uma pilha na computação é imaginar uma pilha física de pratos num refeitório. Se quiser guardar um prato lavado, a única opção lógica é colocá-lo no topo. Se quiser tirar um prato, vai tirar sempre o que está no topo. É impossível tentar tirar o prato que está no fundo sem retirar todos os pratos que estão por cima primeiro.

No desenvolvimento de um jogo 2D na Unity, por exemplo, a interface do usuário (UI) funciona exatamente assim. O jogador abre o "Menu Principal" (fundo da pilha), depois clica em "Configurações" (meio) e depois em "Áudio" (topo). Ao clicar no botão "Voltar", o sistema precisa fechar obrigatoriamente a tela de "Áudio" primeiro. O último a entrar é o primeiro a sair.

A Implementação em C (A Ilusão do Topo) Em C, não existe uma "pilha" mágica. Usamos um array (vetor) comum. A mágica acontece porque nós, programadores, criamos uma regra estrita: ignoramos o resto do array e olhamos apenas para uma variável chamada **topo**.

- **Por que o topo começa em -1?** Em C, os arrays começam no índice 0. Se o topo começasse em 0, estaríamos dizendo que já existe um prato na posição 0. Começando em -1, dizemos matematicamente: "a pilha está vazia".
- **Push (Inserir):** Subimos o topo um degrau (**++topo**) e colocamos o valor lá.
- **Pop (Remover):** Pegamos o valor e descemos o topo um degrau (**topo--**). O valor antigo continua fisicamente na memória RAM, mas para o nosso programa, ele "deixou de existir" porque o topo desceu.
- **Stack Underflow:** É o erro que ocorre quando você tenta dar um **pop()** com o topo já em -1 (tentar tirar um prato de uma mesa vazia).

Módulo 2: Filas (Queues)

A Intuição FIFO (First-In, First-Out)

Diferente da pilha, a fila funciona exatamente como a fila do caixa de um supermercado. O primeiro cliente que chega é o primeiro a ser atendido. Quem chega depois, vai para o final.

No contexto de redes e roteamento (como no ecossistema de certificações Cisco CCNA), os roteadores usam Filas para gerenciar os pacotes de dados. Se muitos pacotes chegam ao roteador ao mesmo tempo para serem enviados por um cabo, o roteador os enfileira. O primeiro pacote que chegou é o primeiro a ser transmitido.

A Implementação em C (Dois Ponteiros e a Roda) Como a fila cresce de um lado e diminui do outro, não basta uma variável. Precisamos de duas: **inicio** (quem está sendo atendido) e **fim** (onde o próximo vai entrar).

- **O problema da fila burra:** Se usarmos um array normal, o **inicio** vai andando para a direita conforme atendemos os clientes. As posições 0, 1 e 2 ficam vazias, mas inutilizáveis. Logo o **fim** bate no limite do array, e o programa diz que a fila está cheia, mesmo com vagas lá atrás!
- **A Solução (Buffer Circular):** Fechamos o array num círculo usando a matemática do resto da divisão (**%**). Quando o **fim** chega no limite máximo, ele "dá a volta" e volta para o índice 0, reaproveitando o espaço morto.

Módulo 3: Ponteiros

A Intuição do Endereço Postal

Em linguagens como Python e Java, a gestão de memória é feita de forma invisível. Você cria objetos e a linguagem cria "referências" automáticas. Em C, não há abstração: você lida diretamente com a placa mãe do computador.

Pense em uma variável como sendo uma

Casa

e o valor dela como sendo as

Pessoas

lá dentro. Um

Ponteiro

não é uma casa, ele é um pedaço de papel onde você anotou o CEP e o número da rua (o endereço de memória RAM).

A Implementação em C (Os Operadores & e *)

- **O Operador & (E comercial):** Significa "Onde fica?". Se você tem `int x = 10;`, e usa `&x`, você não recebe o 10. Você recebe o endereço físico (ex: 0x7FFF) de onde o 10 mora.
- **O Operador * (Asterisco):** Significa "Vá até o endereço e mexa lá dentro". Se `ptr` é o papel com o endereço, usar `*ptr = 50;` significa viajar até aquela casa e trocar os moradores por "50".
- **O grande erro:** Fazer `ptr = 100;` sem o asterisco significa rasgar o papel que tinha o endereço correto e escrever "Rua 100". Quando você tentar visitar a Rua 100, o sistema operacional vai barrar e fechar seu programa (Falha de Segmentação).

Módulo 4: Alocação Dinâmica (malloc, calloc, free)

A Intuição do Loteamento de Terra

A memória do seu programa em C é dividida em dois bairros: a

Stack (Pilha)

e o

Heap (Monte)

.

A Stack é como um quarto de hotel temporário. Quando uma função inicia, ela aluga o quarto, coloca as variáveis lá. Quando a função acaba, o quarto é limpo e destruído automaticamente. O Heap é um latifúndio gigante de terra vazia. Ninguém te dá nada lá, a menos que você vá à prefeitura pedir.

A Implementação em C

- **O que é o `malloc` ?** É você indo à prefeitura e dizendo: "Preciso de um terreno de 20 bytes contínuos no Heap". A função vai lá, cerca o terreno e te dá a chave (um ponteiro para o início). Detalhe: o terreno vem com o mato e o lixo de quem estava lá antes (lixo de memória).
- **O que é o `calloc` ?** É igual ao malloc, mas a prefeitura passa um trator e limpa o terreno todo, deixando todos os bytes perfeitamente com valor Zero antes de te dar a chave.
- **O que é o `free` ?** Como você pediu o terreno manualmente no Heap, o Garbage Collector (que existe no Java/Python) não vai te ajudar aqui. Se você não usar `free(meu_ponteiro);` para devolver a terra à prefeitura, o terreno fica trancado. Se seu programa fizer isso repetidas vezes, a memória RAM do computador inteiro acaba (Memory Leak).

Módulo 5: Manipulação de Arquivos e Streams

A Intuição do Encanamento de Água (Streams)

Salvar dados no disco rígido é uma operação incrivelmente lenta comparada à velocidade da memória RAM. Se o C fosse salvar letra por letra diretamente no disco, o computador ia engasgar.

Por isso usamos

Streams (Fluxos)

. Imagine um cano conectando seu programa (caixa d'água) ao disco rígido (balde). Você não manda gota por gota. Você enche um pequeno reservatório intermediário (chamado

Buffer

) na memória RAM. Quando o Buffer enche, a comporta abre e manda toda a água de uma vez só (Flush).

A Implementação em C

- **Abrindo a Válvula (`fopen`)**: Quando usamos `fopen("dados.txt", "w");` , estamos dizendo: "Crie um cano para este arquivo no modo Write (Escrita)". Se usarmos "r" (Leitura) num arquivo que não existe, o sistema devolve `NULL` porque não tem em quem plugar o cano.
- **Gravando dados**: Funções como `fprintf` e `fwrite` jogam a água dentro do Buffer na RAM. A água ainda **não** está no disco!
- **Fechando a Válvula (`fclose`)**: É a função mais crítica. Ela força a "descarga" (flush) de toda a água que estava presa no Buffer direto para o disco rígido e desmonta o cano. Se você esquecer do `fclose` , seu programa fecha e a água do buffer evapora. O arquivo no disco ficará corrompido ou incompleto.